JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ThreadInfo

java.lang.Object
    java.lang.management.ThreadInfo

```
public class ThreadInfo
extends Object
```

Thread information. `ThreadInfo` contains the information about a thread including:

General thread information

- Thread ID.
- Name of the thread.
- Whether a thread is a daemon thread

Execution information

- Thread state.
- The object upon which the thread is blocked due to:
    - waiting to enter a synchronization block/method, or
    - waiting to be notified in a `Object.wait` method, or
    - parking due to a `LockSupport.park` call.
- The ID of the thread that owns the object that the thread is blocked.
- Stack trace of the thread.
- List of object monitors locked by the thread.
- List of ownable synchronizers locked by the thread.
- Thread priority

Synchronization Statistics

- The number of times that the thread has blocked for synchronization or waited for notification.
- The accumulated elapsed time that the thread has blocked for synchronization or waited for notification since `thread contention monitoring` was enabled. Some Java virtual machine implementation may not support this. The `ThreadMXBean.isThreadContentionMonitoringSupported()` method can be used to determine if a Java virtual machine supports this.

This thread information class is designed for use in monitoring of the system, not for synchronization control.

MXBean Mapping

`ThreadInfo` is mapped to a `CompositeData` with attributes as specified in the `from` method.

Since:

1.5

See Also:

`ThreadMXBean.getThreadInfo(long)`, `ThreadMXBean.dumpAllThreads(boolean, boolean)`

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
`static ThreadInfo`	**`from`**`(CompositeData cd)`	Returns a `ThreadInfo` object represented by the given `CompositeData`.
`long`	**`getBlockedCount`**`()`	Returns the total number of times that the thread associated with this `ThreadInfo` blocked to enter or reenter a monitor.
`long`	**`getBlockedTime`**`()`	Returns the approximate accumulated elapsed time (in milliseconds) that the thread associated with this `ThreadInfo` has blocked to enter or reenter a monitor since thread contention monitoring is enabled.
`MonitorInfo[]`	**`getLockedMonitors`**`()`	Returns an array of `MonitorInfo` objects, each of which represents an object monitor currently locked by the

		thread associated with this `ThreadInfo`.
`LockInfo[]`	**`getLockedSynchronizers()`**	Returns an array of `LockInfo` objects, each of which represents an ownable synchronizer currently locked by the thread associated with this `ThreadInfo`.
`LockInfo`	**`getLockInfo()`**	Returns the `LockInfo` of an object for which the thread associated with this `ThreadInfo` is blocked waiting.
`String`	**`getLockName()`**	Returns the `string representation` of an object for which the thread associated with this `ThreadInfo` is blocked waiting.
`long`	**`getLockOwnerId()`**	Returns the ID of the thread which owns the object for which the thread associated with this `ThreadInfo` is blocked waiting.
`String`	**`getLockOwnerName()`**	Returns the name of the thread which owns the object for which the thread associated with this `ThreadInfo` is blocked waiting.
`int`	**`getPriority()`**	Returns the thread priority of the thread

		associated with this `ThreadInfo`.
`StackTraceElement[]`	**`getStackTrace`**`()`	Returns the stack trace of the thread associated with this `ThreadInfo`.
`long`	**`getThreadId`**`()`	Returns the ID of the thread associated with this `ThreadInfo`.
`String`	**`getThreadName`**`()`	Returns the name of the thread associated with this `ThreadInfo`.
`Thread.State`	**`getThreadState`**`()`	Returns the state of the thread associated with this `ThreadInfo`.
`long`	**`getWaitedCount`**`()`	Returns the total number of times that the thread associated with this `ThreadInfo` waited for notification.
`long`	**`getWaitedTime`**`()`	Returns the approximate accumulated elapsed time (in milliseconds) that the thread associated with this `ThreadInfo` has waited for notification since thread contention monitoring is enabled.
`boolean`	**`isDaemon`**`()`	Tests if the thread associated with this `ThreadInfo` is a daemon thread.
`boolean`	**`isInNative`**`()`	Tests if the thread associated with this `ThreadInfo` is executing native code.

boolean	**isSuspended**()	Tests if the thread associated with this ThreadInfo is suspended.
String	**toString**()	Returns a string representation of this thread info.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Method Details

getThreadId

```
public long getThreadId()
```

Returns the ID of the thread associated with this ThreadInfo.

Returns:

the ID of the associated thread.

getThreadName

```
public String getThreadName()
```

Returns the name of the thread associated with this ThreadInfo.

Returns:

the name of the associated thread.

getThreadState

```
public Thread.State getThreadState()
```

Returns the state of the thread associated with this ThreadInfo.

Returns:

Thread.State of the associated thread.

getBlockedTime

```
public long getBlockedTime()
```

Returns the approximate accumulated elapsed time (in milliseconds) that the thread associated with this `ThreadInfo` has blocked to enter or reenter a monitor since thread contention monitoring is enabled. I.e. the total accumulated time the thread has been in the `BLOCKED` state since thread contention monitoring was last enabled. This method returns `-1` if thread contention monitoring is disabled.

The Java virtual machine may measure the time with a high resolution timer. This statistic is reset when the thread contention monitoring is re-enabled.

Returns:

the approximate accumulated elapsed time in milliseconds that a thread entered the `BLOCKED` state; `-1` if thread contention monitoring is disabled.

Throws:

`UnsupportedOperationException` - if the Java virtual machine does not support this operation.

See Also:

`ThreadMXBean.isThreadContentionMonitoringSupported()`, `ThreadMXBean.setThreadContentionMonitoringEnabled(boolean)`

getBlockedCount

```
public long getBlockedCount()
```

Returns the total number of times that the thread associated with this `ThreadInfo` blocked to enter or reenter a monitor. I.e. the number of times a thread has been in the `BLOCKED` state.

Returns:

the total number of times that the thread entered the `BLOCKED` state.

getWaitedTime

```
public long getWaitedTime()
```

Returns the approximate accumulated elapsed time (in milliseconds) that the thread associated with this `ThreadInfo` has waited for notification since thread contention monitoring is enabled. I.e. the total accumulated time the thread has been in the `WAITING` or `TIMED_WAITING` state since thread contention monitoring is enabled. This method returns `-1` if thread contention monitoring is disabled.

The Java virtual machine may measure the time with a high resolution timer. This statistic is reset when the thread contention monitoring is re-enabled.

Returns:

the approximate accumulated elapsed time in milliseconds that a thread has been in the `WAITING` or `TIMED_WAITING` state; `-1` if thread contention monitoring is disabled.

Throws:

`UnsupportedOperationException` - if the Java virtual machine does not support this operation.

See Also:

`ThreadMXBean.isThreadContentionMonitoringSupported()`, `ThreadMXBean.setThreadContentionMonitoringEnabled(boolean)`

getWaitedCount

```
public long getWaitedCount()
```

Returns the total number of times that the thread associated with this `ThreadInfo` waited for notification. I.e. the number of times that a thread has been in the `WAITING` or `TIMED_WAITING` state.

Returns:

the total number of times that the thread was in the `WAITING` or `TIMED_WAITING` state.

getLockInfo

```
public LockInfo getLockInfo()
```

Returns the `LockInfo` of an object for which the thread associated with this `ThreadInfo` is blocked waiting. A thread can be blocked waiting for one of the following:

- an object monitor to be acquired for entering or reentering a synchronization block/method.
  The thread is in the `BLOCKED` state waiting to enter the `synchronized` statement or method.
- an object monitor to be notified by another thread.
  The thread is in the `WAITING` or `TIMED_WAITING` state due to a call to the `Object.wait` method.
- a synchronization object responsible for the thread parking.
  The thread is in the `WAITING` or `TIMED_WAITING` state due to a call to the `LockSupport.park` method. The synchronization object is the object returned from `LockSupport.getBlocker` method. Typically it is an ownable synchronizer or a `Condition`.

This method returns `null` if the thread is not in any of the above conditions.

Returns:

`LockInfo` of an object for which the thread is blocked waiting if any; `null` otherwise.

Since:

1.6

getLockName

```
public String getLockName()
```

Returns the `string representation` of an object for which the thread associated with this `ThreadInfo` is blocked waiting. This method is equivalent to calling:

```
getLockInfo().toString()
```

This method will return `null` if this thread is not blocked waiting for any object or if the object is not owned by any thread.

Returns:

the
string representation of the object on which the thread is blocked if any; `null` otherwise.

See Also:

`getLockInfo()`

getLockOwnerId

```
public long getLockOwnerId()
```

Returns the ID of the thread which owns the object for which the thread associated with this `ThreadInfo` is blocked waiting. This method will return `-1` if this thread is not blocked waiting for any object or if the object is not owned by any thread.

Returns:

the thread ID of the owner thread of the object this thread is blocked on; `-1` if this thread is not blocked or if the object is not owned by any thread.

See Also:

`getLockInfo()`

getLockOwnerName

```
public String getLockOwnerName()
```

Returns the name of the thread which owns the object for which the thread associated with this `ThreadInfo` is blocked waiting. This method will return `null` if this thread is not blocked waiting for any object or if the object is not owned by any thread.

Returns:

the name of the thread that owns the object this thread is blocked on; `null` if this thread is not blocked or if the object is not owned by any thread.

See Also:

`getLockInfo()`

getStackTrace

```
public StackTraceElement[] getStackTrace()
```

Returns the stack trace of the thread associated with this `ThreadInfo`. If no stack trace was requested for this thread info, this method will return a zero-length array. If the returned array is of non-zero length then the first element of the array represents the top of the stack, which is the most recent method invocation in the sequence. The last element of the array represents the bottom of the stack, which is the least recent method invocation in the sequence.

Some Java virtual machines may, under some circumstances, omit one or more stack frames from the stack trace. In the extreme case, a virtual machine that has no stack trace information concerning the thread associated with this `ThreadInfo` is permitted to return a zero-length array from this method.

Returns:

an array of `StackTraceElement` objects of the thread.

isSuspended

```
public boolean isSuspended()
```

Tests if the thread associated with this `ThreadInfo` is suspended.

Returns:

`true` if the thread is suspended; `false` otherwise.

External Specifications

JVM Tool Interface

isInNative

```
public boolean isInNative()
```

Tests if the thread associated with this `ThreadInfo` is executing native code.

A thread is considered to be executing native code when it is executing a native method, executing native code invoked using a method handle[RESTRICTED] obtained from the native linker, or is attached to the VM with the Java Native Interface with no Java frames on its stack.

Native code does not include Java virtual machine support code, compiled code generated by the Java virtual machine, or special cases such as a thread blocked on a JVM TI raw monitor.

Returns:

`true` if the thread is executing native code; `false` otherwise.

isDaemon

```
public boolean isDaemon()
```

Tests if the thread associated with this `ThreadInfo` is a daemon thread.

Returns:

`true` if the thread is a daemon thread, `false` otherwise.

Since:

9

See Also:

`Thread.isDaemon()`

getPriority

```
public int getPriority()
```

Returns the thread priority of the thread associated with this `ThreadInfo`.

Returns:

The priority of the thread associated with this `ThreadInfo`.

Since:

9

toString

```
public String toString()
```

Returns a string representation of this thread info. The format of this string depends on the implementation. The returned string will typically include the thread name, the thread ID, its state, and a stack trace if any.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this thread info.

from

```
public static ThreadInfo from(CompositeData cd)
```

Returns a `ThreadInfo` object represented by the given `CompositeData`. A `CompositeData` representing a `ThreadInfo` of version *N* must contain all of the attributes defined in version ≤ *N* unless specified otherwise. The same rule applies the composite type of the given `CompositeData` and transitively to attributes whose type or component type is `CompositeType`.

A `CompositeData` representing `ThreadInfo` of version *N* contains `"stackTrace"` attribute and `"lockedMonitors"` attribute representing an array of `StackTraceElement` and an array of `MonitorInfo` respectively and their types are of version *N*. The `"lockedStackFrame"` attribute in `MonitorInfo`'s composite type must represent `StackTraceElement` of the same version *N*. Otherwise, this method will throw `IllegalArgumentException`.

Attribute Name	Type	Since
threadId	`java.lang.Long`	5
threadName	`java.lang.String`	5
threadState	`java.lang.String`	5
suspended	`java.lang.Boolean`	5
inNative	`java.lang.Boolean`	5
blockedCount	`java.lang.Long`	5
blockedTime	`java.lang.Long`	5
waitedCount	`java.lang.Long`	5
waitedTime	`java.lang.Long`	5
lockName	`java.lang.String`	5
lockOwnerId	`java.lang.Long`	5
lockOwnerName	`java.lang.String`	5
stackTrace	`javax.management.openmbean.CompositeData[]`, each element is a `CompositeData` representing `StackTraceElement` as specified below.	5
lockInfo	`javax.management.openmbean.CompositeData` - the mapped type for `LockInfo` as specified in the `LockInfo.from(CompositeData)` method. If the given `CompositeData` does not contain this attribute, the `LockInfo` object will be	6

Attribute Name	Type	Since
	constructed from the value of the `lockName` attribute.	
lockedMonitors	`javax.management.openmbean.CompositeData[]` whose element type is the mapped type for `MonitorInfo` as specified in the `MonitorInfo.from` method. If the given `CompositeData` does not contain this attribute, this attribute will be set to an empty array.	6
lockedSynchronizers	`javax.management.openmbean.CompositeData[]` whose element type is the mapped type for `LockInfo` as specified in the `LockInfo.from(CompositeData)` method. If the given `CompositeData` does not contain this attribute, this attribute will be set to an empty array.	6
daemon	`java.lang.Boolean` If the given `CompositeData` does not contain this attribute, this attribute will be set to `false`.	9
priority	`java.lang.Integer` If the given `CompositeData` does not contain this attribute, This attribute will be set to `Thread.NORM_PRIORITY`.	9

A `CompositeData` representing `StackTraceElement` of version N must contain all of the attributes defined in version $\leq N$ unless specified otherwise.

Attribute Name	Type	Since
classLoaderName	`java.lang.String`	9
moduleName	`java.lang.String`	9
moduleVersion	`java.lang.String`	9
className	`java.lang.String`	5
methodName	`java.lang.String`	5
fileName	`java.lang.String`	5
lineNumber	`java.lang.Integer`	5
nativeMethod	`java.lang.Boolean`	5

Parameters:

`cd` - `CompositeData` representing a `ThreadInfo`

Returns:

a `ThreadInfo` object represented by `cd` if `cd` is not `null`; `null` otherwise.

Throws:

`IllegalArgumentException` - if the given `cd` and its composite type does not contain all of the attributes defined for a `ThreadInfo` of a specific runtime version.

getLockedMonitors

```
public MonitorInfo[] getLockedMonitors()
```

Returns an array of `MonitorInfo` objects, each of which represents an object monitor currently locked by the thread associated with this `ThreadInfo`. If no locked monitor was requested for this thread info or no monitor is locked by the thread, this method will return a zero-length array.

Returns:

an array of `MonitorInfo` objects representing the object monitors locked by the thread.

Since:

1.6

getLockedSynchronizers

```
public LockInfo[] getLockedSynchronizers()
```

Returns an array of `LockInfo` objects, each of which represents an ownable synchronizer currently locked by the thread associated with this `ThreadInfo`. If no locked synchronizer was requested for this thread info or no synchronizer is locked by the thread, this method will return a zero-length array.

Returns:

an array of `LockInfo` objects representing the ownable synchronizers locked by the thread.

Since:

1.6

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other

countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.